

รายงานการพัฒนาเว็บแอปพลิเคชัน Social Media
ด้วย React และ Firebase Authentication

จัดทำโดย

นาย จิรวัฒน์ ดอนบรรเทา รหัสนักศึกษา 69419010003

เสนอโดย

อาจารย์ นางสาวศรารัตน์ วรรณแจ่ม

รายวิชา 694190101-26-41901-2104-การพัฒนาซอฟต์แวร์ด้วยเทคโนโลยี Front-End

แผนกวิชา เทคโนโลยีสารสนเทศ วิทยาลัยเทคนิคขอนแก่น

ภาคเรียนที่ 1 ปีการศึกษา 2569

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่ออธิบายแนวคิด โครงสร้าง และการทำงานของโครงการ Nexora Social Community ซึ่งเป็นเว็บไซต์ชุมชนออนไลน์ที่พัฒนาด้วยเทคโนโลยี Front-End ผู้ใช้สามารถอ่านโพสต์ ค้นหาข้อมูล สร้างโพสต์ แสดงความคิดเห็น ตอบกลับ และดู Story ได้จากหน้าเดียวกัน

โครงการใช้ Firebase Authentication สำหรับสมัครสมาชิก เข้าสู่ระบบ และเข้าสู่ระบบด้วย Google รายงานจึงอธิบายทั้งมุมมองผู้ใช้และมุมมองการทำงานของโค้ด โดยใช้คำที่ตรงกับสิ่งที่มีอยู่จริงในโครงการ

เนื้อหาส่วนปัญหาและวิธีแก้ไขเพิ่มตัวอย่างที่ผู้เริ่มต้นมักพบ เช่น การลิมิเปิดผู้ให้บริการใน Firebase การตั้งค่าโดเมนสำหรับ Google Sign-In เพื่อให้ใช้เป็นแนวทางพัฒนาต่อไป

จิรวัดน์ ดอนบรรเทา

15 สิงหาคม 2569

สารบัญ

คำนำ.....	2
1. ขอบเขตของระบบ	4
2. เครื่องมือที่ใช้	4
3. โครงสร้างโปรเจกต์	5
4. ขั้นตอนการสมัครสมาชิก	6
5. ขั้นตอนการ Login.....	7
6. การทำงานของ Firebase Authentication	7
7. การทำงานของ UserAuthContext	8
8. การทำงานของ ProtectedRoute.....	8
9. การทำงานของ Navbar.....	8
10. การสร้างโพสต์.....	9
11. การออกแบบ CSS	9
12. ภาพหน้าจอการทำงานของระบบ	10
13. ปัญหาและวิธีแก้ไข	12
14. สรุปผลการพัฒนา.....	13
15. ข้อเสนอแนะ	13

1. ขอบเขตของระบบ

Nexora Social Community เป็นเว็บแบบ Single-page application หรือเว็บที่เปลี่ยนเนื้อหาภายในหน้าเดิมตามเส้นทางที่ผู้ใช้เปิดจุดประสงค์ของโครงการคือฝึกพัฒนา เว็บไซต์ที่มีหน้าตาและการโต้ตอบคล้ายพื้นที่ชุมชนออนไลน์ โดยเน้นการทำงานฝั่งผู้ใช้เป็นหลัก

สิ่งที่ผู้ใช้ทำได้ในเวอร์ชันปัจจุบัน

- ดู Feed, Story, รายชื่อผู้ติดต่อ และข้อมูลเดโมในหน้าแรก
- ค้นหาข้อความโพสต์ รายชื่อผู้ติดต่อ และ Group Chats ด้วยคำไทยหรืออังกฤษ
- สร้าง ลบ แชร์ และแสดงความรู้สึกกับโพสต์ รวมถึงเขียนความคิดเห็นและตอบกลับ
- สร้าง Story จากข้อความหรือรูปภาพที่ผ่านการตรวจชนิดไฟล์และขนาด
- สมัครสมาชิกและเข้าสู่ระบบด้วยอีเมล รหัสผ่าน หรือบัญชี Google ผ่าน Firebase Authentication

ข้อจำกัดของเวอร์ชัน demo: โพสต์ ความคิดเห็น และ Story เก็บอยู่ใน state ของเบราว์เซอร์ จึงหายไปเมื่อรีเฟรชหน้าเว็บ ระบบยังไม่มี Firestore, Firebase Storage หรือฐานข้อมูลถาวร

2. เครื่องมือที่ใช้

เครื่องมือแต่ละตัวมีหน้าที่ต่างกัน โครงการจึงแยกส่วนสำหรับสร้างหน้าจอ จัดการเส้นทาง ยืนยันตัวตน และตรวจคุณภาพโค้ดออกจากกัน

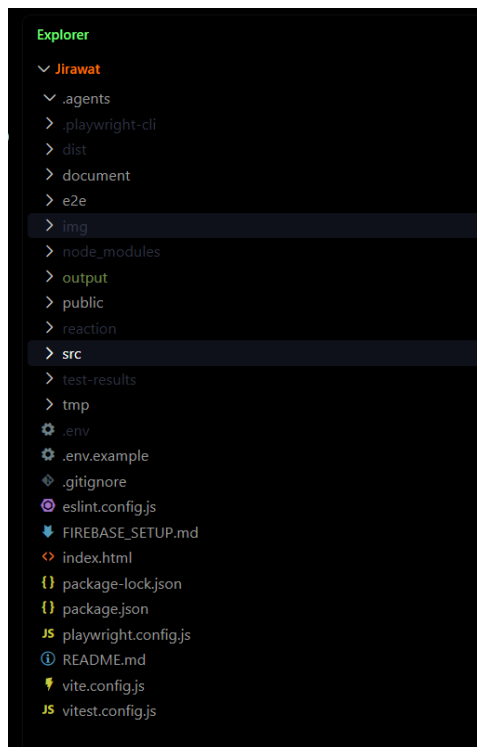
เครื่องมือ	ใช้ทำอะไร
React 19 และ React DOM	สร้างส่วนประกอบของหน้าจอและจัดการข้อมูลที่เปลี่ยนแปลงตามการใช้งาน
Vite 8	ใช้เปิดเว็บระหว่างพัฒนา และสร้างไฟล์สำหรับนำไปเผยแพร่
React Router DOM	กำหนดเส้นทางหน้าแรก หน้าเข้าสู่ระบบ และหน้าสมาชิก
Firebase Authentication	สมัครสมาชิก เข้าสู่ระบบ ออกจากระบบ และเข้าสู่ระบบด้วย Google
React Icons และ PropTypes	แสดงไอคอนและช่วยตรวจสอบรูปแบบข้อมูลที่ส่งให้ component

เครื่องมือ	ใช้ทำอะไร
ESLint, Vitest, Testing Library และ Playwright	ตรวจสอบคุณภาพโค้ด ทดสอบหน่วยย่อย และทดสอบการใช้งานผ่านเบราว์เซอร์
Git และ GitHub	เก็บประวัติการพัฒนาและเผยแพร่ source code

3. โครงสร้างโปรเจกต์

ไฟล์ในโครงการแบ่งตามหน้าที่ เพื่อให้ค้นหาและแก้ไขได้ง่าย โดยส่วนที่ผู้ใช้งานมองเห็นอยู่ใน components และ pages ส่วนข้อมูลและตรรกะรวมอยู่ใน data, context, hooks และ utils

- src/main.jsx เริ่มต้นแอป สร้าง Router และครอบทุกหน้าด้วย UserAuthContextProvider
- src/App.jsx วางโครงหน้าหลัก รับคำค้นจาก Navbar และส่งต่อไปยัง Home
- src/pages/Home.jsx เก็บ state ของ Feed, post, comment, reply, reaction และ modal
- src/components/ รวมชิ้นส่วนหน้าจอ เช่น Navbar, Login, Signup, Post, Story และ Sidebar
- src/context/ เก็บ UserAuthContext และ custom hook useUserAuth สำหรับสถานะการเข้าสู่ระบบ
- src/hooks/ เก็บ logic ที่ใช้ซ้ำ เช่น การส่งฟอร์ม authentication และการจัดการ dialog
- src/data/ และ src/constants.js เก็บข้อมูลเดโม รายชื่อผู้ติดต่อ Story และข้อมูล reaction
- src/utils/ เก็บตัวช่วย เช่น imageUpload.js และ authError.js
- src/styles/ และ src/App.css เก็บ design token และ CSS แยกตามส่วนของหน้าจอ
- document/ เก็บเอกสารอธิบายโครงการและภาพหน้าจอ ส่วน e2e/ เก็บกรณีทดสอบผ่านเบราว์เซอร์



รูปภาพ โครงสร้างโปรเจกต์

4. ขั้นตอนการสมัครสมาชิก

ผู้ใช้เปิดหน้า /signup เพื่อสร้างบัญชีใหม่ หน้า Signup มีช่องอีเมล รหัสผ่าน และยืนยันรหัสผ่าน พร้อมปุ่มสำหรับเข้าสู่ระบบด้วย Google

1. กรอกอีเมล รหัสผ่าน และยืนยันรหัสผ่าน หรือเลือกปุ่มดำเนินการต่อด้วย Google
2. ระบบตรวจสอบรูปแบบอีเมล ตรวจสอบว่ารหัสผ่านมีอย่างน้อย 6 ตัวอักษร และตรวจสอบว่ารหัสผ่านสองช่องตรงกัน
3. เมื่อข้อมูลผ่านการตรวจสอบ หน้า Signup เรียก signUp จาก UserAuthContext
4. UserAuthContext ส่งอีเมลและรหัสผ่านให้ Firebase ผ่าน createUserWithEmailAndPassword
5. เมื่อ Firebase ยืนยันสำเร็จ ระบบพากลับไปหน้า / และ Navbar เปลี่ยนเป็นสถานะผู้ที่เข้าสู่ระบบแล้ว

5. ขั้นตอนการ Login

ผู้ใช้เปิดหน้า /login เพื่อเข้าสู่ระบบ หน้า Login ออกแบบเป็น Nexora Aurora Portal โดยมีส่วนแนะนำแบรนด์อยู่ด้านซ้ายบนหน้าจอใหญ่ และมี form อยู่ด้านขวา เมื่อใช้มือ ถือจะเหลือเฉพาะส่วนที่จำเป็นต่อการเข้าสู่ระบบ

1. กรอกอีเมลและรหัสผ่าน หรือเลือกปุ่มดำเนินการต่อด้วย Google
2. LoginForm ตรวจสอบข้อมูลก่อนส่ง และมีปุ่มแสดงหรือซ่อนรหัสผ่านเพื่อลดการพิมพ์ผิด
3. ระหว่างรอ Firebase ตอบกลับ ปุ่มจะอยู่ในสถานะกำลังทำงาน เพื่อป้องกันการกดส่งซ้ำ
4. หน้า Login เรียก login หรือ loginWithGoogle จาก FirebaseAuthContext
5. เมื่อเข้าสู่ระบบสำเร็จ ระบบพากลับหน้าแรก

หากไม่สำเร็จจะแสดงข้อความภาษาไทยที่บอกผู้ใช้งานว่าควรตรวจสอบอะไร

6. การทำงานของ Firebase Authentication

ไฟล์ src/firebase.js อ่านค่า Firebase Web configuration จากตัวแปรที่ขึ้นต้นด้วย VITE_FIREBASE_ ในไฟล์ .env ก่อนเริ่มใช้งาน หากค่าตั้งต้นไม่ครบ ระบบจะไม่สร้าง auth object และคืนข้อความผิดพลาดที่อธิบายได้

เมื่อค่าครบ ระบบใช้ initializeApp เพียงครั้งเดียว โดยตรวจ app ที่มีอยู่ก่อนเพื่อป้องกันการเริ่ม Firebase ซ้ำ ระหว่างการสมัครสมาชิกหรือเข้าสู่ระบบ Firebase จะจัดการการยืนยันตัวตน และ session ของผู้ใช้

- Email/Password ใช้ createUserWithEmailAndPassword สำหรับสมัคร และ signInWithEmailAndPassword สำหรับเข้าสู่ระบบ
- Google Sign-In ใช้ GoogleAuthProvider และ signInWithPopup
- การออกจากระบบใช้ signOut
- การติดตาม session ใช้ onAuthStateChanged เพื่อให้หน้าจอรู้อาใครกำลังเข้าสู่ระบบอยู่

ความปลอดภัย: ไฟล์ .env ไม่ควรนำขึ้น GitHub โครงการมี .env.example

ไว้บอกชื่อค่าที่ต้องตั้งในเครื่องใหม่ และควรจำกัด Authorized domains ใน Firebase ก่อน deploy

7. การทำงานของ UserAuthContext

UserAuthContext คือที่เก็บสถานะการเข้าสู่ระบบร่วมกันของทั้งแอป แทนที่จะส่งข้อมูลผู้ใช้ผ่าน props หลายชั้น หน้า Login, Signup และ Navbar จึงเรียก useUserAuth เพื่อใช้ข้อมูลชุดเดียวกันได้

- user เก็บข้อมูลผู้ใช้จาก Firebase หรือเป็น null เมื่อยังไม่ได้เข้าสู่ระบบ
- loading บอกว่า Firebase กำลังตรวจ session เพื่อไม่ให้ Navbar แสดงปุ่มผิดสถานะระหว่างเริ่มแอป
- signUp, login, loginWithGoogle และ logout เป็นฟังก์ชันที่ component เรียกใช้
- useEffect ลงทะเบียน onAuthStateChanged หนึ่งครั้ง และคืน unsubscribe เพื่อหยุด listener เมื่อ provider ถูกปิด

custom hook useUserAuth จะตรวจว่าถูกเรียกภายใน UserAuthContextProvider หากใช้ผิดตำแหน่งจะ throw error เพื่อให้ผู้พัฒนาหาสาเหตุได้เร็ว

8. การทำงานของ ProtectedRoute

จากการตรวจ source code ไม่พบ component หรือ route ชื่อ ProtectedRoute ในเวอร์ชันปัจจุบัน นี่ไม่ใช่ข้อผิดพลาด เพราะหน้า Home ตั้งใจเป็นพื้นที่สาธารณะที่ผู้เข้าชมดู Feed และ Story ได้แม้ยังไม่ได้เข้าสู่ระบบ

หากพัฒนาต่อและมีหน้า profile, settings หรือข้อมูลส่วนตัว ควรเพิ่ม ProtectedRoute เพื่อบังคับให้ loading เสร็จ ตรวจ user และพาไปหน้า /login เมื่อไม่มี session การตัดสินใจนี้ควรวางคู่กับกติกาสิทธิ์ของข้อมูลใน backend

9. การทำงานของ Navbar

Navbar ทำหน้าที่เป็นแถบหลักของเว็บ มีโลโก้ Nexora ช่องค้นหา เมนูกลาง และพื้นที่สถานะผู้ใช้ โดยช่องค้นหาเป็น controlled input ที่รับ searchQuery และ onSearchChange จาก App เพื่อให้ Home และ ContactsSidebar ใช้ค่าค้นเดียวกัน

- ก่อนเข้าสู่ระบบจะแสดงปุ่ม Login และ Sign Up
- หลังเข้าสู่ระบบจะแสดงข้อมูลโปรไฟล์เดโมและปุ่ม Logout
- ใช้ค่า loading จาก UserAuthContext เพื่อป้องกันปุ่ม Login และ Sign Up กระพริบผิดสถานะ

- เมนูที่ยังเป็น demo ใช้ disabled และข้อความกำกับ เพื่อไม่ทำให้ผู้ใช้เข้าใจว่ากดใช้งานได้
- CSS responsive ซ่อนเมนูกลางในช่วง tablet และย่อโลโก้กับช่องค้นหาในหน้าจอมือถือ

10. การสร้างโพสต์

Home.jsx เป็นจุดรวม state ของ Feed ผู้ใช้พิมพ์ข้อความหรือเลือกภาพจาก Input component แล้วส่งข้อมูลไปยัง addPost ระบบจะไม่สร้างโพสต์ว่าง และจะเพิ่ม โพสต์ใหม่ไว้ด้านบนของ Feed

1. เลือกสร้างโพสต์จากกล่อง composer
2. หากเลือกภาพ imageUpload.js จะตรวจชนิดไฟล์ ขนาดไม่เกิน 5 MB และตรวจ magic bytes ก่อนอ่านไฟล์ด้วย FileReader
3. Home สร้าง id แบบเพิ่มขึ้นด้วย useRef เพื่อลดโอกาส key ซ้ำระหว่าง session
4. ผู้ใช้สามารถกด reaction, เพิ่ม comment, ตอบ reply, share post หรือเปิด PostModal เพื่อดูรายละเอียด
5. Copy Link แสดง toast ว่าเป็นฟีเจอร์ demo เพราะยังไม่มี URL ของโพสต์จริง จึงไม่สร้างลิงก์ปลอม

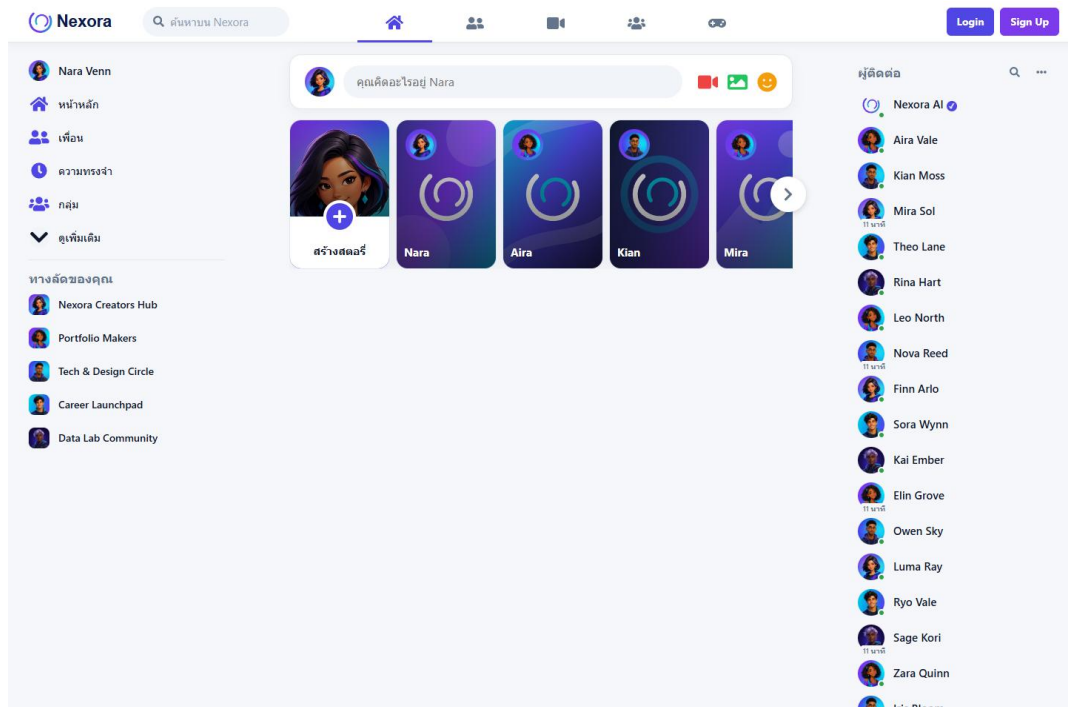
11. การออกแบบ CSS

CSS ของ Nexora แยกตามส่วนที่รับผิดชอบและรวมผ่าน App.css ไฟล์ variables.css เป็นจุดกลางของ design tokens เช่น สี ระยะขอบ เงาม และ focus ring ทำให้ส่วนต่าง ๆ ใช้รูปแบบใกล้เคียงกันและแก้ริ้วได้จากจุดเดียว

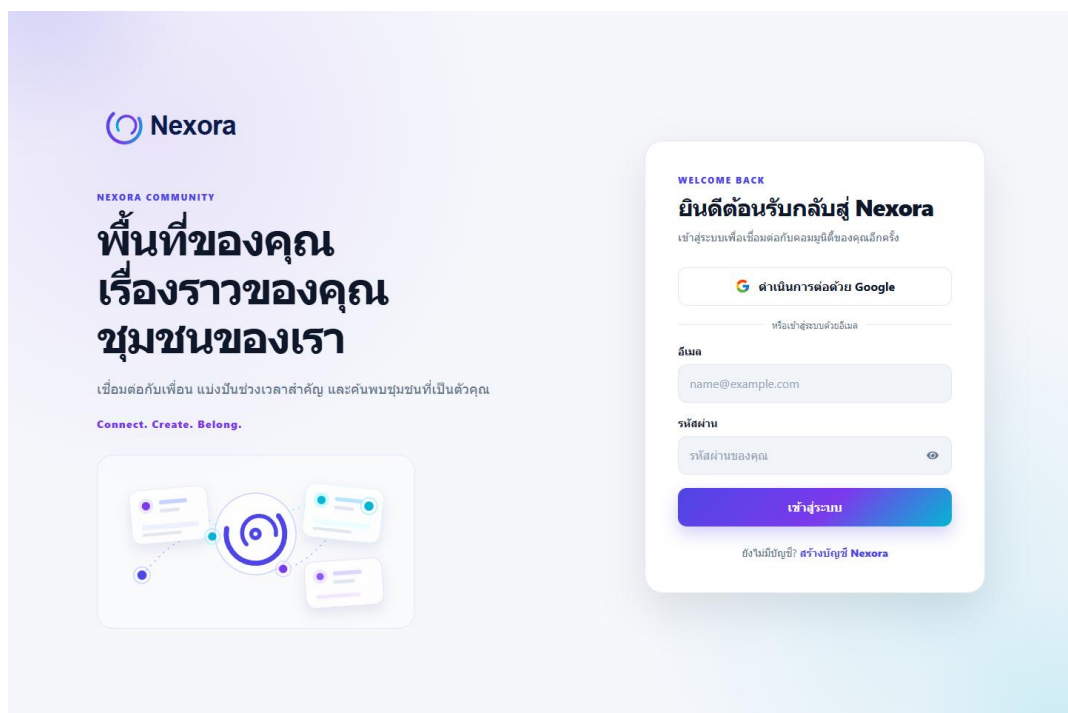
- ใช้สีหลัก Indigo #4F46E5, Violet #7C3AED และ Cyan #06B6D4 เพื่อให้ภาพลักษณ์เป็น Nexora
- ใช้ surface สีขาว พื้นหลังอ่อน ขอบโค้ง และเงาเบาเพื่อให้ card อ่านง่าย
- กำหนด :focus-visible ให้ปุ่ม ช่องกรอก และลิงก์ เพื่อช่วยการใช้งานด้วยคีย์บอร์ด
- แยก responsive CSS สำหรับ desktop, tablet และ mobile โดยซ่อน sidebar หรือ contacts เมื่อพื้นที่ไม่พอ
- หน้า Login และ Signup ใช้ Aurora Portal และภาพ Community Constellation เพื่อให้แบรนด์สม่ำเสมอ

12. ภาพหน้าจอการทำงานของระบบ

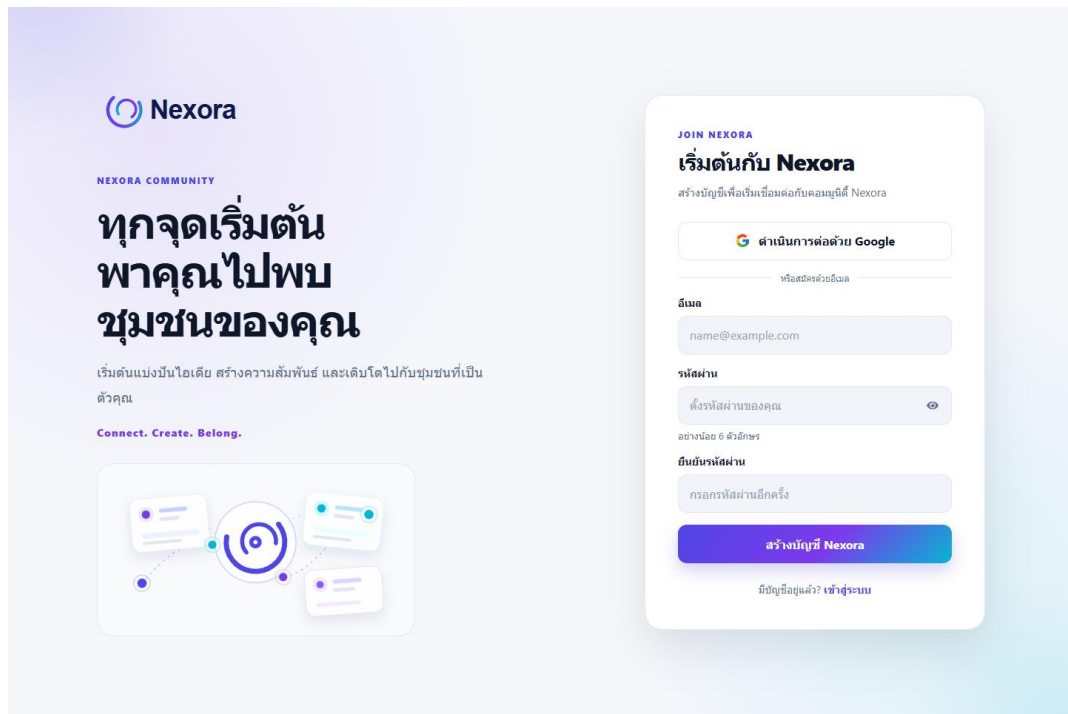
ภาพต่อไปนี้จะใช้ข้อมูลเดโมในโครงการและแสดงหน้าจอสำคัญที่ผู้ใช้พบระหว่างใช้งาน



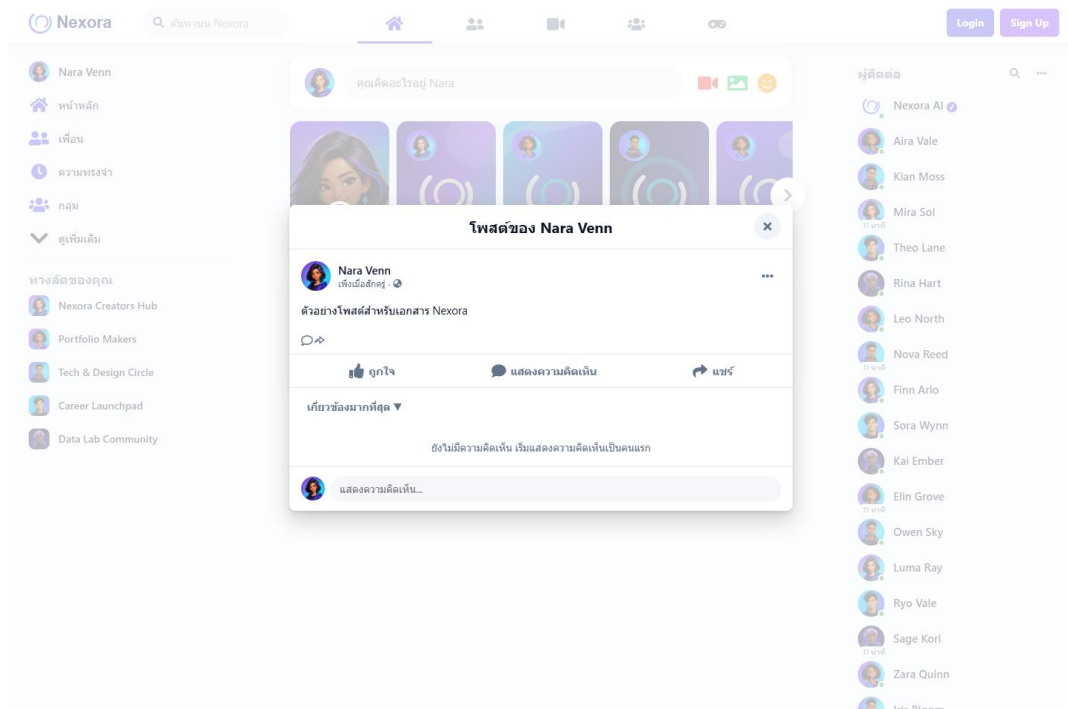
ภาพที่ 1 หน้า Home แสดง Feed, Story, Sidebar และ Contacts



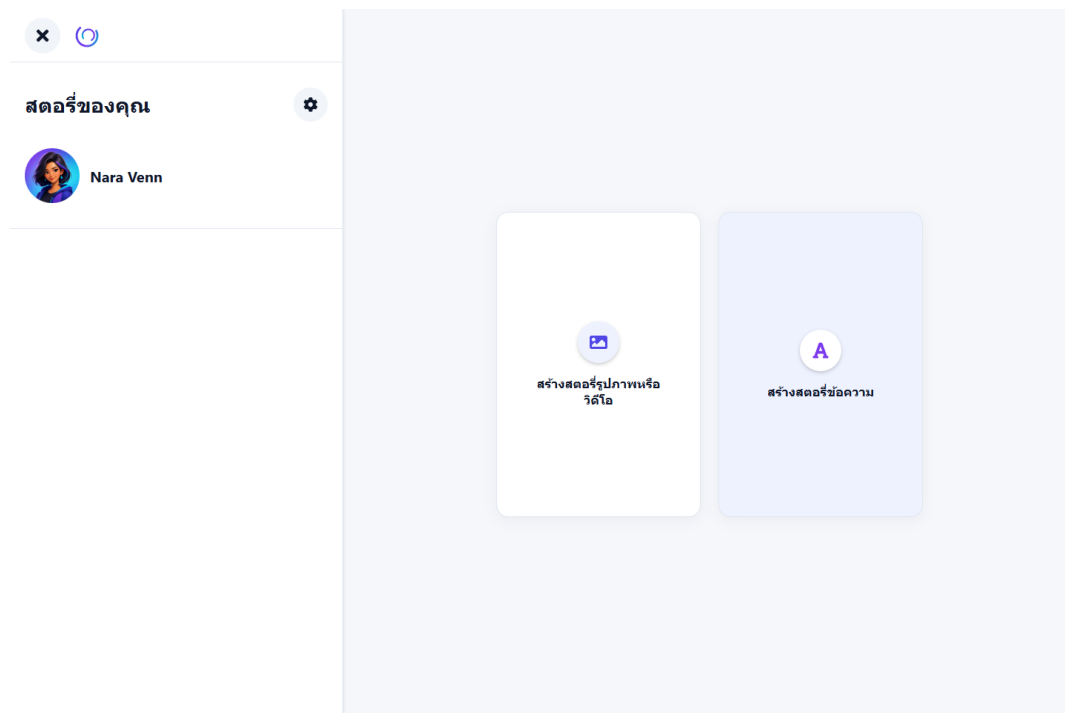
ภาพที่ 2 หน้า Login ของ Nexora Aurora Portal



ภาพที่ 3 หน้า Signup สำหรับสมัครด้วยอีเมลหรือ Google



ภาพที่ 4 Post Modal สำหรับอ่านรายละเอียดและเพิ่มความคิดเห็น



ภาพที่ 5 หน้า Story Creator สำหรับสร้าง Story

13. ปัญหาและวิธีแก้ไข

หัวข้อนี้แยกทั้งข้อจำกัดของโครงงานและปัญหาพื้นฐานที่ผู้เริ่มต้นมักพบ เพื่อให้ตรวจสอบได้ทีละจุดก่อนเปลี่ยนโค้ด

ปัญหา	วิธีตรวจและแก้ไข
สมัครสมาชิกหรือเข้าสู่ระบบด้วยอีเมลไม่ได้	ตรวจ Firebase Console > Authentication > Sign-in method แล้วเปิด Email/Password จากนั้นตรวจอีเมลรหัสผ่าน และข้อความแจ้งเตือนในหน้า Login หรือ Signup
กด Google Sign-In แล้วหน้าต่างไม่เปิดหรือถูกปฏิเสธ	ตรวจว่าเปิด Google provider แล้ว เลือก Project support email อนุญาต popup ในเบราว์เซอร์ และเพิ่ม localhost หรือโดเมนที่ deploy ใน Authorized domains
อัปโหลดรูปไม่ได้	รองรับ JPEG, PNG, WebP และ GIF เท่านั้น ขนาดต้องไม่เกิน 5 MB หากเป็นไฟล์จากมือถือให้ตรวจนามสกุลและขนาดก่อนเลือก

ปัญหา	วิธีตรวจและแก้ไข
ข้อมูลโพสต์หายเมื่อรีเฟรช	เป็นข้อจำกัดของเวอร์ชัน demo เพราะเก็บข้อมูลไว้ใน state ของเบราว์เซอร์ ต้องเพิ่ม Firestore หรือฐานข้อมูลก่อนใช้งานจริง
ต้องการซ่อนหน้าที่เป็นข้อมูลส่วนตัว	เวอร์ชันนี้ยังไม่มี ProtectedRoute เพราะหน้า Home ตั้งใจให้ทุกคนดูได้ หากเพิ่ม profile หรือ settings ต้องสร้าง route protection และกำหนดสิทธิ์ให้ชัดเจน

14. สรุปผลการพัฒนา

Nexora Social Community ทำตามเป้าหมายของโครงการ Front-End ได้ครบในส่วน of หน้าเว็บชุมชนออนไลน์ ผู้ใช้ดูข้อมูลเดโม ค้นหาโพสต์ สร้างและโต้ตอบกับโพสต์ ใช้ Story และสมัครหรือเข้าสู่ระบบผ่าน Firebase ได้

โครงสร้าง component, context, hooks, data และ CSS แยกหน้าที่ชัดเจน จึงเหมาะกับการอ่านโค้ดและต่อยอดเป็นระบบที่ใหญ่ขึ้น ผลตรวจล่าสุดผ่าน lint, unit tests, production build และ Playwright E2E อย่างไรก็ตาม dependency audit ยังมีประเด็น React Router ที่ต้องจัดการก่อนนำระบบไปใช้จริง

15. ข้อเสนอแนะ

หากต้องพัฒนาต่อจากโครงการ demo ไปสู่ระบบใช้งานจริง ควรทำตามลำดับต่อไปนี้

1. อัปเดต React Router ตาม advisory ตรวจ lockfile และรัน npm run test:verify ก่อน merge ทุกครั้ง
2. เพิ่ม Firestore พร้อม Security Rules เพื่อบันทึกโพสต์ ความคิดเห็น และข้อมูล profile แบบถาวร
3. เพิ่ม Firebase Storage พร้อมกติกากำหนด ชนิดไฟล์ และสิทธิ์การเข้าถึงสำหรับรูปภาพ
4. เพิ่ม ProtectedRoute สำหรับหน้า profile หรือ settings และกำหนดสิทธิ์ใน backend ไม่ใช่เฉพาะหน้าเว็บ
5. เพิ่ม reset password, email verification และหน้าแก้ไขโปรไฟล์
6. ก่อน deploy ควรมี Terms, Privacy Policy, ระบบรายงานเนื้อหา การติดตาม error และขั้นตอน CI/CD ที่แยก environment ชัดเจน